

Lista de exercícios — Revisão de Lista, Pilha e Fila

Estruturas base em C:

Lista simplesmente encadeada

```
typedef struct NoLista {  
    int valor;  
    struct NoLista* prox;  
} NoLista;
```

```
typedef struct {  
    NoLista* inicio;  
    int tamanho;  
} Lista;
```

Pilha encadeada

```
typedef struct NoPilha {  
    int valor;  
    struct NoPilha* prox;  
} NoPilha;
```

```
typedef struct {  
    NoPilha* topo;  
    int tamanho;  
} Pilha;
```

Fila encadeada

```
typedef struct NoFila {  
    int valor;  
    struct NoFila* prox;  
} NoFila;
```

```
typedef struct {  
    NoFila* inicio;  
    NoFila* fim;  
    int tamanho;  
} Fila;
```

1. Durante um plantão, os alunos chegam para atendimento técnico e entram em ordem de chegada. Considerando essa situação, responda:

- (a) Qual estrutura representa melhor esse comportamento: lista, pilha ou fila?
- (b) Qual é a política de acesso dessa estrutura?
- (c) Em uma implementação dinâmica, por que guardar também o ponteiro para o fim pode ser útil?

2. Uma pilha está inicialmente vazia e recebe as operações abaixo:

```
push(14), push(9), push(31), pop(), push(5), pop(), push(18)
```

Responda:

- (a) Qual elemento está no topo ao final?
- (b) Em que ordem os elementos restantes seriam removidos?
- (c) Qual elemento foi removido primeiro?

3. Uma fila de senhas começa vazia e recebe, nessa ordem:

```
enqueue(40), enqueue(12), dequeue(), enqueue(27), enqueue(33),  
dequeue(), enqueue(51)
```

Qual será a sequência de remoção dos elementos restantes?

- (a) 27, 33, 51
- (b) 12, 27, 51
- (c) 33, 27, 51
- (d) 27, 51, 33

4. Classifique cada afirmação em **V** ou **F**.

- I. Em uma lista simplesmente encadeada, cada nó conhece apenas o próximo nó.
- II. Em uma pilha, inserção e remoção acontecem em extremidades diferentes.
- III. Em uma fila encadeada, a remoção ocorre no início e a inserção normalmente ocorre no fim.
- IV. Uma lista encadeada dinâmica exige posições contíguas de memória para seus nós.

5. O objetivo da função abaixo é inserir um novo valor no início da lista.

```
void inserirInicio(Lista* lista, int valor) {  
    NoLista* novo = (NoLista*) malloc(sizeof(NoLista));  
    novo->valor = valor;  
    lista->inicio = novo;  
    novo->prox = lista->inicio;  
    lista->tamanho++;  
}
```

Faça:

- (a) Identifique o erro principal.
- (b) Explique a consequência na estrutura.
- (c) Reescreva apenas as linhas necessárias para corrigir.

6. A função abaixo deve inserir um elemento no topo da pilha.

```
void push(Pilha* pilha, int valor) {  
    NoPilha* novo = (NoPilha*) malloc(sizeof(NoPilha));  
    novo->valor = valor;  
  
    /* COMPLETE AQUI */  
  
    pilha->tamanho++;  
}
```

Escreva o trecho faltante.

7. A função abaixo deve remover o primeiro elemento da fila e retorná-lo. Se a fila estiver vazia, deve retornar -1.

```
int desenfileirar(Fila* fila) {  
    if (fila->inicio == NULL) {  
        return -1;  
    }  
  
    NoFila* removido = fila->inicio;  
    int valor = removido->valor;  
  
    /* COMPLETE AQUI */  
  
    return valor;  
}
```

Observe o trecho de código acima e escreva o trecho que falta para completar a função.

Lembre-se do caso em que a fila fica vazia após a remoção.

8. Uma coleção de valores sofre as transformações abaixo:

$$8 \rightarrow 84 \rightarrow 841 \rightarrow 84 \rightarrow 847$$

O elemento removido foi o **último** que havia sido inserido.

Qual estrutura está sendo representada?

- (a) Fila
- (b) Pilha
- (c) Lista circular
- (d) Árvore

9. Considere o código abaixo, que tenta implementar a função de remoção de uma pilha.

```
int pop(Pilha* pilha) {
    if (pilha->topo == NULL) {
        return -1;
    }

    NoPilha* removido = pilha->topo;
    int valor = removido->valor;
    free(removido);
    pilha->topo = removido->prox;
    pilha->tamanho--;
    return valor;
}
```

Faça:

- (a) Aponte o erro conceitual no código.
- (b) Explique por que ele ocorre.
- (c) Corrija com a menor alteração possível.

10. Associe cada descrição à estrutura mais adequada.

Situação	Estrutura
Desfazer a última ação em um editor de texto	
Guardar uma sequência genérica em que posso percorrer nó a nó	
Atender clientes por ordem de chegada	

Preencha com: lista, pilha ou fila.

11. Observe a seguinte função de inserção em fila.

```
void enfileirar(Fila* fila, int valor) {
    NoFila* novo = (NoFila*) malloc(sizeof(NoFila));
    novo->valor = valor;
    novo->prox = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = novo;
    } else {
        fila->fim->prox = novo;
    }

    fila->tamanho++;
}
```

Responda:

- (a) O que falta para a função funcionar corretamente em todos os casos?
- (b) Qual problema aparece na segunda inserção, caso isso não seja corrigido?

12. Considere a lista abaixo:

$11 \rightarrow 4 \rightarrow 18 \rightarrow 7 \rightarrow \text{NULL}$

Deseja-se remover o nó que contém o valor 18.

Responda:

- (a) Qual nó deve ter seu ponteiro `prox` alterado?
- (b) Para qual nó ele passará a apontar?
- (c) O que acontece se o nó removido for liberado sem antes ajustar o encadeamento?

13. A função abaixo deve retornar 1 se o valor estiver na lista e 0 caso contrário.

```
int contem(Lista* lista, int valor) {  
    NoLista* atual = lista->inicio;  
  
    while (atual != NULL) {  
        /* COMPLETE AQUI */  
        atual = atual->prox;  
    }  
  
    return 0;  
}
```

Observe o trecho de código acima e escreva o trecho que falta para completar a função.

14. Um aluno afirmou:

“Como lista, pilha e fila são feitas com nós e ponteiros, então as três estruturas são iguais; muda apenas o nome das funções.”

Explique por que essa afirmação está **incorreta**. Em sua resposta, comente o papel das **regras de acesso** da estrutura.

15. Considere a função de remoção de uma fila com apenas um elemento.

- (a) Após remover esse único nó, como devem ficar `inicio` e `fim`?
- (b) Por que deixar `fim` apontando para memória já liberada é perigoso?

16. Em um sistema de impressão, os documentos enviados vão sendo colocados ao final da estrutura, e a impressora sempre pega o documento mais antigo ainda não impresso.

A alternativa correta é:

- (a) O comportamento descrito é de uma pilha, pois o último arquivo enviado é o primeiro impresso.
- (b) O comportamento descrito é de uma fila, pois há atendimento em ordem de chegada.
- (c) O comportamento descrito é de uma lista, pois listas sempre removem o primeiro elemento.
- (d) O comportamento descrito é de uma pilha, pois a inserção ocorre no fim.

17. Complete a função abaixo, responsável por inserir um novo valor no fim da fila. Lembre-se de considerar o caso em que a fila está inicialmente vazia.

```

void enfileirar(Fila* fila, int valor) {
    NoFila* novo = (NoFila*) malloc(sizeof(NoFila));
    novo->valor = valor;
    novo->prox = NULL;

    if (fila->inicio == NULL) {
        /* COMPLETE AQUI */
    } else {
        /* COMPLETE AQUI */
    }

    fila->tamanho++;
}

```

18. Uma playlist foi armazenada dinamicamente na forma abaixo:

(21) → (8) → (34) → (13) → NULL

Deseja-se inserir o valor 50 no **início** da lista.

Responda:

- (a) Como ficará o novo encadeamento?
- (b) Qual ponteiro do novo nó deve receber o antigo *inicio*?
- (c) Quantos nós existirão após a inserção?

19. Responda apenas com base na ideia de manipulação de ponteiros:

- (a) Em uma pilha encadeada, por que inserir e remover no topo é natural?
- (b) Em uma fila encadeada, por que ter os ponteiros *inicio* e *fim* simplifica as operações básicas?
- (c) Em uma lista simplesmente encadeada, por que remover o último nó tende a exigir mais cuidado do que remover o primeiro?

20. O código abaixo tenta implementar uma função que esvazia uma pilha.

```

void limparPilha(Pilha* pilha) {
    NoPilha* atual = pilha->topo;

    while (atual != NULL) {
        free(atual);
        atual = atual->prox;
    }

    pilha->tamanho = 0;
}

```

Faça:

- (a) Identifique o erro conceitual no laço.
- (b) Explique por que ele pode causar comportamento indefinido.

(c) Reescreva apenas o corpo do `while` corretamente.

(d) Ao final, além de `tamanho = 0`, qual outro ajuste deve ser feito na estrutura?